

OpenStack Cloud Portal

Platform Deployment Specification

High Availability | Security | Compliance & Governance
Scalability | Operational Excellence | BCDR

Document Version:	1.0
Date:	April 08, 2026
Classification:	Confidential
Status:	Draft for Review
Author:	Platform Engineering Team

Table of Contents

1. **Executive Summary**
2. **Platform Architecture Overview**
3. **High Availability (HA) Design**
4. **Security Architecture**
5. **Compliance & Governance**
6. **Scalability Design**
7. **Operational Excellence**
8. **Business Continuity & Disaster Recovery (BCDR)**
9. **Admin Module - Isolated Deployment**
10. **Network Architecture & Segmentation**
11. **Monitoring, Logging & Observability**
12. **CI/CD Pipeline & Release Management**
13. **Cost Management & FinOps**
14. **Deployment Environments & Promotion**
15. **Deployment Runbook & Procedures**
16. **Appendices**

1. Executive Summary

This document provides the comprehensive deployment specification for the OpenStack Cloud Portal, a multi-tenant cloud management and billing platform. The platform enables organizations to manage OpenStack cloud infrastructure through a unified web interface with full lifecycle management, billing, and administrative capabilities.

The specification covers all aspects of a production-grade deployment including High Availability, Security, Compliance & Governance, Scalability, Operational Excellence, and Business Continuity & Disaster Recovery (BCDR). A key architectural decision is the separation of the Admin Module Portal from the Customer-facing Portal to ensure platform management operations are isolated, secure, and independently scalable.

1.1 Platform Components

The platform consists of the following microservices:

- **API Gateway (Node.js/Express):** Central entry point with JWT validation, rate limiting, and request routing (Port 3000)
- **Auth Service (Node.js/Express):** Authentication, user management, JWT generation, Keystone integration (Port 3001)
- **Admin Service (Python/FastAPI):** Project management, quota management, audit logging (Port 8001)
- **Cloud Service (Python/FastAPI):** OpenStack SDK wrapper for 18+ cloud services (Port 8002)
- **Billing Service (Node.js/Express):** Usage tracking, invoice generation, plan-based pricing (Port 3002)
- **Notification Service (Node.js):** Async RabbitMQ consumer, email notifications (Port 3003)
- **Frontend (React/TypeScript/Vite):** Unified web portal served via Nginx (Port 80)

1.2 Infrastructure Dependencies

- **PostgreSQL 15:** Primary relational data store for all services
- **Redis 7:** Rate limiting, session cache, token blacklist
- **RabbitMQ 3:** Asynchronous inter-service messaging
- **OpenStack (Keystone, Nova, Neutron, Cinder, Glance, etc.):** Backend cloud infrastructure

2. Platform Architecture Overview

2.1 Deployment Topology

The platform is deployed on Kubernetes (K8s) using a namespace-isolated microservices architecture. All services are containerized using Docker with multi-stage builds for optimized image sizes. The deployment targets a minimum of 3-node Kubernetes cluster for production workloads with separate node pools for application and data services.

Target Deployment Architecture:

- Kubernetes Cluster: Minimum 3 master nodes + 5 worker nodes (production)
- Namespace: 'openstack-portal' for customer portal, 'openstack-admin' for admin portal
- Ingress: NGINX Ingress Controller with TLS termination
- Service Mesh: Istio (recommended) for mTLS, traffic management, observability
- Container Runtime: containerd
- Container Registry: Private registry (Harbor or cloud-native registry)

2.2 Two-Portal Architecture

CRITICAL DESIGN DECISION: The platform is deployed as two separate portals with distinct network boundaries, access controls, and scaling profiles:

1) Customer Portal (openstack-portal namespace):

- Frontend, API Gateway, Auth Service, Cloud Service, Billing Service, Notification Service
- Accessible via public-facing ingress (e.g., portal.example.com)
- Handles tenant operations, resource provisioning, billing

2) Admin Portal (openstack-admin namespace):

- Admin Frontend, Admin API Gateway, Admin Service, Auth Service (admin-only)
- Accessible only via internal/VPN ingress (e.g., admin.internal.example.com)
- Handles platform management, quota management, audit, user administration

2.3 Data Flow Architecture

Request flow: Client -> CDN/WAF -> Load Balancer -> Ingress Controller -> API Gateway -> Backend Services -> Data Stores / OpenStack APIs. Async events: Services -> RabbitMQ -> Notification Service -> SMTP/Webhooks.

3. High Availability (HA) Design

3.1 Application Tier HA

All stateless microservices are deployed with a minimum of 2 replicas across different availability zones using Kubernetes pod anti-affinity rules. Horizontal Pod Autoscalers (HPA) ensure automatic scaling based on CPU/memory utilization.

Service	Min Replicas	Max Replicas	CPU Target	Anti-Affinity
API Gateway	3	10	70%	Zone-spread
Auth Service	3	8	70%	Zone-spread
Cloud Service	3	8	70%	Zone-spread
Billing Service	2	6	70%	Zone-spread
Notification Service	2	4	70%	Zone-spread
Frontend (Nginx)	3	6	70%	Zone-spread
Admin API Gateway	2	4	70%	Zone-spread
Admin Service	2	4	70%	Zone-spread

3.2 Database Tier HA

PostgreSQL HA Configuration:

- Primary-Standby replication using Patroni or CloudNativePG operator
- Minimum 3-node cluster: 1 primary + 2 synchronous standbys
- Automatic failover with < 30 second RTO
- Connection pooling via PgBouncer (min 2 replicas)
- WAL archiving to object storage for point-in-time recovery (PITR)
- Shared storage or replicated PVs across availability zones

Redis HA Configuration:

- Redis Sentinel or Redis Cluster with 3+ nodes
- Automatic failover with master election
- Persistence: AOF + RDB snapshots for data durability
- Dedicated Redis instances for rate limiting vs. session cache

RabbitMQ HA Configuration:

- 3-node RabbitMQ cluster with quorum queues
- Mirrored queues across all nodes
- Automatic partition handling (pause_minority mode)
- Persistent message storage with acknowledgments

3.3 Ingress & Load Balancing HA

- Dual NGINX Ingress Controller replicas across availability zones
- External Load Balancer (L4) in front of ingress controllers
- Health check endpoints on all services (/health, /ready)
- Circuit breaker pattern implemented at API Gateway level
- Graceful degradation: billing and notification services can be temporarily unavailable without impacting core cloud operations

3.4 HA Targets

Component	Availability Target	RTO	RPO
Customer Portal	99.95%	< 5 min	0 (stateless)
Admin Portal	99.9%	< 10 min	0 (stateless)
PostgreSQL	99.99%	< 30 sec	< 1 sec (sync)
Redis	99.9%	< 60 sec	< 5 sec
RabbitMQ	99.9%	< 60 sec	0 (persistent)
Overall Platform	99.95%	< 5 min	< 1 min

4. Security Architecture

4.1 Authentication & Authorization

Multi-layered authentication and authorization framework:

- **JWT-based Authentication:** All API requests authenticated via JWT tokens issued by Auth Service. Tokens include user role, project scope, and expiration (configurable, default 24h for customer, 8h for admin)
- **Keystone Integration:** Auth Service delegates identity verification to OpenStack Keystone for federated identity management
- **Role-Based Access Control (RBAC):** Three-tier role model: Super Admin, Project Admin, User. Enforced at API Gateway and individual service levels
- **Token Blacklisting:** Revoked tokens stored in Redis with TTL matching token expiration
- **MFA (Multi-Factor Authentication):** Required for Admin Portal access; recommended for Customer Portal

4.2 Network Security

- Network Policies: Kubernetes NetworkPolicy restricting pod-to-pod communication to only required paths
- Service Mesh mTLS: All inter-service communication encrypted via Istio mutual TLS
- Ingress TLS: TLS 1.3 termination at ingress with automated cert rotation (cert-manager + Let's Encrypt or enterprise CA)
- WAF: Web Application Firewall (ModSecurity or cloud-native WAF) in front of ingress
- DDoS Protection: Rate limiting at API Gateway (configurable per-endpoint, default 100 req/min per IP)
- Admin Portal: Accessible only via VPN or private network - NEVER exposed to public internet
- Egress Control: Restrict outbound traffic to only required destinations (OpenStack APIs, SMTP, DNS)

4.3 Data Security

- **Encryption at Rest:** PostgreSQL volumes encrypted using storage-class encryption (LUKS or cloud KMS). Redis AOF/RDB files on encrypted PVs
- **Encryption in Transit:** All traffic encrypted via TLS 1.3 (external) and mTLS (internal)
- **Secrets Management:** Kubernetes Secrets backed by external vault (HashiCorp Vault or cloud KMS). No plaintext secrets in ConfigMaps or environment variables
- **Database Security:** Separate database credentials per service (principle of least privilege). Connection encryption enforced (sslmode=verify-full)
- **PII Handling:** Customer PII encrypted at application level. Audit logs do not contain sensitive data. Log scrubbing for credentials

4.4 Container Security

- Base Images: Minimal, distroless or Alpine-based images. No root user in containers
- Image Scanning: Trivy or Snyk scanning in CI/CD pipeline. Block deployment of images with CRITICAL/HIGH CVEs
- Pod Security Standards: Enforce 'restricted' PSS profile. Read-only root filesystem where possible
- Resource Limits: CPU/memory limits enforced on all pods to prevent resource abuse
- Image Provenance: Signed images with cosign. Admission controller (Kyverno/OPA) enforcing image policies

4.5 API Security

- Rate Limiting: Per-IP and per-user rate limits at API Gateway (express-rate-limit)
- Input Validation: Server-side validation on all endpoints (express-validator, pydantic)
- CORS: Strict origin whitelist - only portal domains allowed
- CSRF Protection: Anti-CSRF tokens for state-changing operations
- Security Headers: HSTS, X-Content-Type-Options, X-Frame-Options, CSP headers via Nginx
- API Versioning: All APIs versioned (/api/v1/) to enable safe deprecation

5. Compliance & Governance

5.1 Regulatory Compliance Framework

The platform deployment must satisfy applicable regulatory and industry compliance requirements. The following frameworks are considered:

Framework	Scope	Key Requirements
SOC 2 Type II	All services	Access controls, audit logging, encryption, change management
ISO 27001	Platform infrastructure	ISMS, risk assessment, security controls
GDPR	Customer data handling	Data minimization, consent, right to erasure, DPA
PCI DSS	Billing service	Cardholder data protection, network segmentation, logging
CIS Benchmarks	Kubernetes, OS	Hardened configurations for K8s, containers, OS

5.2 Audit Logging & Trail

Comprehensive audit logging is implemented across all layers:

- **Application Audit:** Admin Service maintains detailed audit log of all administrative actions (project CRUD, quota changes, user management) stored in PostgreSQL with immutable append-only design
- **API Audit:** API Gateway logs all requests with correlation ID, user identity, action, resource, timestamp, source IP
- **Infrastructure Audit:** Kubernetes audit logging enabled for all API server requests. Node-level audit via auditd
- **Retention:** Audit logs retained for minimum 1 year (hot: 90 days in Elasticsearch, warm: 365 days in object storage)
- **Tamper Protection:** Logs shipped to centralized logging in real-time. Write-once object storage for long-term retention

5.3 Governance Policies

- **Change Management:** All changes to production require approved PR, passing CI/CD, and deployment approval
- **Access Governance:** Quarterly access reviews. JIT (Just-In-Time) access for administrative operations
- **Configuration Drift:** GitOps-based deployment ensures declared state matches running state. Drift detection alerts
- **Vulnerability Management:** Weekly automated scans. SLA: Critical < 24h, High < 7d, Medium < 30d
- **Incident Management:** Defined severity levels (P1-P4) with escalation procedures and communication templates

- Data Governance: Data classification policy (Public, Internal, Confidential, Restricted). Handling procedures per classification

5.4 Quota & Resource Governance

The Admin Service enforces resource quotas per project/tenant. This prevents resource hoarding and ensures fair utilization:

Resource	Default Quota	Max Quota	Governance
vCPUs	20	500	Admin approval required above default
RAM (GB)	50	2048	Admin approval required above default
Volumes	10	200	Admin approval for > 50
Volume Storage (GB)	500	10000	Capacity planning review for > 2TB
Floating IPs	5	50	Network team approval for > 20
Networks	5	50	Network team approval for > 10
Security Groups	10	100	Auto-approved

6. Scalability Design

6.1 Horizontal Scaling

All application services are stateless and horizontally scalable via Kubernetes HPA. The scaling strategy follows a tiered approach:

- **Tier 1 - Automatic HPA:** CPU/memory-based autoscaling for predictable load patterns. Configured via HPA manifests for all stateless services
- **Tier 2 - Custom Metrics HPA:** Scale based on request queue depth (RabbitMQ), active connections, or request latency using KEDA or Prometheus Adapter
- **Tier 3 - Cluster Autoscaler:** Automatically add/remove Kubernetes worker nodes based on pending pod demands
- **Tier 4 - Multi-Cluster:** For large-scale deployments, use federation or multi-cluster service mesh for geographic distribution

6.2 Database Scaling

- Read Replicas: PostgreSQL read replicas for read-heavy workloads (billing queries, audit log reads)
- Connection Pooling: PgBouncer with transaction-level pooling to handle high connection counts
- Partitioning: Time-based partitioning for audit_logs and billing_usage tables
- Archival: Automated archival of data older than retention period to cold storage

6.3 Caching Strategy

- Redis L1 Cache: Token validation cache, rate limit counters, session data (TTL: 5-60 min)
- API Response Cache: Frequently accessed, slowly changing data (flavors, images list) cached at API Gateway
- CDN: Static frontend assets served via CDN with cache-busting versioned filenames
- OpenStack API Caching: Cloud Service caches OpenStack catalog and quota info (TTL: 5 min)

6.4 Capacity Planning

Scale Tier	Concurrent Users	K8s Workers	PostgreSQL	Redis	RabbitMQ
Small	< 100	3 nodes	2 vCPU / 8GB	2GB	2GB
Medium	100 - 1,000	5 nodes	4 vCPU / 16GB	4GB	4GB
Large	1,000 - 10,000	10 nodes	8 vCPU / 32GB	8GB	8GB
Enterprise	> 10,000	20+ nodes	16 vCPU / 64GB (cluster)	16GB (cluster)	16GB (cluster)

7. Operational Excellence

7.1 GitOps Deployment Model

All infrastructure and application configurations are managed as code in Git repositories. Deployments are automated and declarative using GitOps principles:

- Infrastructure as Code: Kubernetes manifests in /k8s/base/ with Kustomize overlays for environments
- GitOps Operator: ArgoCD or Flux for continuous reconciliation of desired vs. actual state
- Branching Strategy: trunk-based development with short-lived feature branches
- Environment Promotion: dev -> staging -> production via Git merge and automated pipelines

7.2 Health Checks & Readiness

Every service implements standardized health check endpoints:

Probe Type	Endpoint	Interval	Threshold	Purpose
Liveness	/health	30s	3 failures	Restart unhealthy containers
Readiness	/ready	10s	2 failures	Remove from load balancer
Startup	/health	5s	30 failures	Allow slow startup

7.3 Configuration Management

- Non-sensitive config: Kubernetes ConfigMaps (DB host, service URLs, feature flags)
- Sensitive config: Kubernetes Secrets backed by HashiCorp Vault or external KMS
- Per-environment overrides: Kustomize overlays for dev/staging/production
- Config hot-reload: Services watch for ConfigMap changes and reload without restart where possible

7.4 Runbook Automation

- Automated scaling scripts (/scripts/scale.sh) for manual intervention scenarios
- Database backup/restore procedures automated and tested monthly
- Certificate rotation automated via cert-manager with 30-day pre-expiry renewal
- Secret rotation procedures with zero-downtime rolling updates

7.5 SLA & SLO Definitions

SLI (Indicator)	SLO (Objective)	SLA (Agreement)	Measurement
Availability	99.95%	99.9%	Uptime Robot / synthetic checks
API Latency (p99)	< 500ms	< 1s	Prometheus histogram
Error Rate	< 0.1%	< 0.5%	5xx response ratio

Time to First Byte

< 200ms

< 500ms

CDN/ingress metrics

Deployment Frequency

> 10/week

N/A

CI/CD pipeline metrics

Change Failure Rate

< 5%

< 10%

Rollback tracking

MTTR

< 30 min

< 1 hour

Incident tracking

8. Business Continuity & Disaster Recovery (BCDR)

8.1 Backup Strategy

Component	Backup Method	Frequency	Retention	Storage
PostgreSQL	pg_dump + WAL archiving	Full: daily, WAL: continuous	30 days full, 7 days WAL	S3/object storage (cross-region)
Redis	RDB snapshots + AOF	RDB: hourly, AOF: continuous	7 days	S3/object storage
RabbitMQ	Definition export + msg persistence	Definitions: daily	30 days	S3/object storage
K8s Config	Velero cluster backup	Daily	90 days	S3/object storage (cross-region)
Secrets/Vault	Vault snapshot	Daily	90 days	Encrypted S3 (separate account)
Container Images	Registry replication	On push	Indefinite (tagged)	Secondary registry

8.2 Disaster Recovery Tiers

DR strategy follows a tiered approach based on criticality:

Tier 1: Hot Standby (RTO < 15 min, RPO < 1 min)

- PostgreSQL synchronous replication to DR site
- Active-passive Kubernetes cluster in secondary AZ/region
- DNS failover via health-check based routing (Route53/CloudFlare)
- Applicable to: Customer Portal, Auth Service, Cloud Service

Tier 2: Warm Standby (RTO < 1 hour, RPO < 15 min)

- PostgreSQL async replication to DR site
- Pre-provisioned but scaled-down K8s cluster in DR region
- Applicable to: Billing Service, Notification Service

Tier 3: Cold Recovery (RTO < 4 hours, RPO < 24 hours)

- Restore from backups to a new environment
- Infrastructure provisioned via IaC (Terraform/Kustomize)
- Applicable to: Admin Portal (can tolerate longer downtime)

8.3 DR Testing

- Tabletop exercises: Quarterly

- Failover testing: Semi-annually (with controlled failover to DR site)
- Full DR simulation: Annually (simulate complete primary site loss)
- Chaos engineering: Monthly (pod failure, node failure, network partition tests via LitmusChaos)

8.4 Business Continuity Plan

- Communication Plan: Defined escalation matrix, status page (e.g., Statuspage.io), customer notification templates
- Decision Authority: On-call engineer can initiate failover; DR declaration requires VP Engineering approval
- Recovery Procedures: Documented step-by-step runbooks for each DR tier in /docs/runbooks/
- Post-Incident: Blameless post-mortems within 48 hours of any P1/P2 incident

9. Admin Module - Isolated Deployment

9.1 Separation Rationale

The Admin Module Portal is deployed as a completely separate instance from the Customer Portal. This architectural decision is driven by the following requirements:

- Security Isolation: Admin operations (user management, quota changes, audit access) must be protected by additional security layers not applicable to customer traffic
- Blast Radius Reduction: A security incident or DDoS on the customer portal must not impact admin operations
- Compliance: Regulatory frameworks require separation of duties and access controls for administrative functions
- Independent Scaling: Admin portal has different traffic patterns (low volume, high privilege) vs. customer portal (high volume, standard privilege)
- Maintenance Windows: Admin portal can be updated independently without customer-facing downtime

9.2 Admin Portal Architecture

The Admin Portal consists of:

Component	Technology	Deployment	Access
Admin Frontend	React/TS (admin-only routes)	Separate Nginx deployment	VPN/internal only
Admin API Gateway	Node.js/Express	Separate deployment with admin-specific middleware	VPN/internal only
Admin Service	Python/FastAPI	Dedicated namespace 'openstack-admin'	Internal cluster network
Auth Service (Admin)	Node.js/Express	Separate instance with admin-only auth flow	Internal cluster network
PostgreSQL	Shared cluster, separate databases	admin_portal_db with restricted credentials	Service accounts only
Redis	Shared cluster, separate keyspace	Prefix: admin:	Service accounts only

9.3 Admin Network Isolation

Network architecture for admin portal isolation:

- Separate Kubernetes namespace: 'openstack-admin' with strict NetworkPolicies
- No direct ingress from public internet - accessible only via VPN gateway or bastion host
- Dedicated ingress controller for admin traffic (not shared with customer ingress)

- NetworkPolicy: Deny all ingress by default, allow only from VPN CIDR range
- mTLS enforced between admin services and shared data stores
- Separate DNS: admin.internal.example.com (internal DNS only, not resolvable externally)

9.4 Admin Access Controls

- VPN Required: All admin portal access requires active VPN connection with certificate-based authentication
- MFA Mandatory: TOTP or hardware key (FIDO2/WebAuthn) required for all admin logins
- Session Management: Admin sessions expire after 8 hours (vs. 24h for customer). Idle timeout: 30 minutes
- IP Allowlisting: Admin API Gateway only accepts requests from known VPN egress IPs
- Privileged Access Management: JIT access elevation for super-admin operations. All elevated access time-boxed and audited
- Separation of Duties: No single admin can both create and approve quota changes above threshold

9.5 Admin Deployment Configuration

Kubernetes deployment specifications for admin portal:

Resource	Admin API GW	Admin Service	Admin Auth	Admin Frontend
CPU Request	100m	150m	100m	50m
CPU Limit	500m	500m	500m	200m
Memory Request	128Mi	256Mi	128Mi	64Mi
Memory Limit	512Mi	512Mi	512Mi	256Mi
Min Replicas	2	2	2	2
Max Replicas	4	4	4	3
HPA CPU Target	70%	70%	70%	70%

10. Network Architecture & Segmentation

10.1 Network Zones

Zone	Purpose	Access	Components
DMZ	Public-facing traffic	Internet via WAF/LB	CDN, WAF, External LB
Application Zone	Customer portal services	From DMZ only	Customer API GW, services, frontend
Admin Zone	Admin portal services	From VPN only	Admin API GW, services, frontend
Data Zone	Databases and caches	From App/Admin zones only	PostgreSQL, Redis, RabbitMQ
Integration Zone	External API access	Controlled egress	OpenStack API endpoints, SMTP
Management Zone	Ops tooling	VPN + bastion	Monitoring, logging, CI/CD runners

10.2 Kubernetes Network Policies

Strict network policies enforced at namespace level:

- Default deny all ingress and egress in both namespaces
- Explicit allow rules for required service-to-service communication paths
- Customer portal services cannot communicate with admin namespace
- Data zone services accept connections only from application and admin namespaces on specific ports
- DNS (CoreDNS) access allowed for all pods for service discovery
- Egress to OpenStack APIs restricted to Cloud Service pods only

10.3 TLS & Certificate Management

- External TLS: cert-manager with Let's Encrypt (staging) or enterprise CA (production)
- Internal mTLS: Istio service mesh with automatic certificate rotation (24h TTL)
- Database TLS: PostgreSQL configured with ssl=on, client certificates for service accounts
- Certificate Monitoring: Alert on certificates expiring within 14 days

11. Monitoring, Logging & Observability

11.1 Monitoring Stack

Three pillars of observability implemented:

- **Metrics:** Prometheus + Grafana. Service-level metrics (request rate, error rate, duration), infrastructure metrics (CPU, memory, disk, network), business metrics (active users, billing events)
- **Logging:** EFK Stack (Elasticsearch, Fluentd, Kibana) or Loki + Grafana. Structured JSON logging from all services. Centralized with correlation IDs for request tracing
- **Tracing:** Jaeger or Tempo for distributed tracing. OpenTelemetry instrumentation in all services for end-to-end request visibility

11.2 Alerting Strategy

Severity	Example	Response Time	Channel	Escalation
P1 - Critical	Platform down, data loss risk	< 15 min	PagerDuty + Phone	VP Eng in 30 min
P2 - High	Service degraded, HA failover	< 30 min	PagerDuty + Slack	Eng Manager in 1h
P3 - Medium	Performance degradation	< 2 hours	Slack + Email	Team lead next standup
P4 - Low	Non-critical warning	Next business day	Email + Ticket	Sprint backlog

11.3 Key Dashboards

- Platform Overview: Service health, request rates, error rates, latency percentiles
- Infrastructure: Node CPU/memory, pod status, PV usage, network I/O
- Database: Connection pool utilization, query performance, replication lag, disk usage
- Security: Failed auth attempts, rate limit hits, WAF blocks, certificate expiry countdown
- Business: Active users, resource provisioning rate, billing revenue, quota utilization
- Admin Portal: Admin action frequency, audit trail activity, quota change requests

12. CI/CD Pipeline & Release Management

12.1 Pipeline Architecture

Multi-stage CI/CD pipeline using GitHub Actions (or GitLab CI):

Build Stage

- Lint: ESLint (Node.js), Ruff/Black (Python), Prettier (frontend)
- Unit Tests: Jest (Node.js), Pytest (Python), React Testing Library (frontend)
- Security Scan: Trivy (container), Snyk (dependencies), SAST (SonarQube)
- Docker Build: Multi-stage builds, image tagging with git SHA + semver
- Image Push: Push to private container registry (Harbor/ECR/GCR)

Deploy Stage

- Dev: Auto-deploy on merge to develop branch
- Staging: Auto-deploy on merge to main branch. Integration tests run post-deploy
- Production: Manual approval gate. Canary deployment (10% -> 50% -> 100%) with automated rollback on error spike

12.2 Release Strategy

- Semantic Versioning: MAJOR.MINOR.PATCH for all services
- Canary Deployments: Gradual traffic shift with automated health validation
- Blue-Green (Admin Portal): Full environment swap for admin portal due to lower traffic
- Database Migrations: Forward-only migrations via Alembic. Backward-compatible schema changes only
- Feature Flags: LaunchDarkly or Unleash for progressive feature rollout
- Rollback: Automated rollback if error rate > 1% during canary. Manual rollback available via ArgoCD

12.3 Artifact Management

- Container Registry: Harbor (self-hosted) or cloud-native registry with vulnerability scanning
- Helm Charts: Versioned Helm charts for each service stored in ChartMuseum or OCI registry
- Build Artifacts: Signed with cosign, SBOMs generated with Syft

13. Cost Management & FinOps

13.1 Resource Optimization

- Right-sizing: Quarterly review of resource requests vs. actual utilization. Adjust CPU/memory accordingly
- Spot/Preemptible Nodes: Use for non-critical workloads (batch billing calculations, notification processing)
- Cluster Autoscaler: Scale-down during off-peak hours. Minimum viable cluster maintained 24/7
- Storage Tiering: Hot data in SSD-backed PVs, cold data (old audit logs, archived invoices) in object storage

13.2 Cost Allocation

- Namespace-level resource quotas and cost tracking
- Kubecost or OpenCost for real-time cost visibility per service/namespace
- Chargeback model: Platform costs allocated to business units based on resource consumption
- Monthly cost reviews with engineering leadership

14. Deployment Environments & Promotion

14.1 Environment Matrix

Environment	Purpose	Data	Access	Deployment
Development	Feature development	Synthetic/seeded	Developers	Auto on branch push
Staging	Pre-production validation	Anonymized prod copy	Dev + QA	Auto on main merge
Production	Live customer traffic	Real	SRE + Ops	Manual approval
DR	Disaster recovery standby	Replicated from prod	SRE only	Continuous replication

14.2 Environment Parity

All environments use identical container images, Kubernetes manifests (with Kustomize overlays for environment-specific config), and deployment procedures. The only differences are:

- Resource allocation (smaller in dev/staging)
- Replica counts (lower in dev/staging)
- External endpoints (different domains, TLS certs)
- Secrets (different credentials per environment)
- Feature flags (gradual rollout across environments)

15. Deployment Runbook & Procedures

15.1 Pre-Deployment Checklist

- [] All CI/CD checks passing (lint, test, security scan)
- [] Container images built, scanned, and signed
- [] Database migrations tested in staging
- [] Rollback plan documented and tested
- [] Change request approved (for production)
- [] Monitoring dashboards reviewed for baseline metrics
- [] On-call engineer notified of deployment window
- [] Customer communication prepared (if user-facing changes)

15.2 Deployment Steps - Customer Portal

- 1. Merge approved PR to main branch
- 2. CI/CD pipeline builds, tests, and pushes images
- 3. ArgoCD detects manifest changes and begins sync
- 4. Canary deployment: 10% traffic to new version
- 5. Monitor error rates, latency for 10 minutes
- 6. If healthy: promote to 50%, monitor 10 minutes
- 7. If healthy: promote to 100%
- 8. If unhealthy at any stage: automatic rollback
- 9. Post-deployment smoke tests (automated)
- 10. Update deployment log and close change request

15.3 Deployment Steps - Admin Portal

- 1. Schedule maintenance window (low-traffic period)
- 2. Notify admin users of upcoming deployment
- 3. Blue-green deployment: deploy new version to inactive environment
- 4. Run admin-specific smoke tests against inactive environment
- 5. Switch traffic to new environment
- 6. Monitor for 15 minutes
- 7. If issues: instant rollback by switching back to previous environment
- 8. Decommission old environment after 24-hour bake period

15.4 Rollback Procedures

- Automated: ArgoCD rollback to previous healthy revision
- Manual: `kubectl rollout undo deployment/<service-name> -n <namespace>`
- Database: Backward-compatible migrations allow rollback without schema revert
- Full: Velero restore from cluster backup (last resort)

16. Appendices

16.1 Appendix A - Port Mapping

Service	Container Port	K8s Service Port	Protocol
API Gateway	3000	3000	HTTP/HTTPS
Auth Service	3001	3001	HTTP
Admin Service	8001	8001	HTTP
Cloud Service	8002	8002	HTTP
Billing Service	3002	3002	HTTP
Notification Service	3003	3003	HTTP
Frontend	80	80	HTTP
PostgreSQL	5432	5432	TCP
Redis	6379	6379	TCP
RabbitMQ	5672 / 15672	5672 / 15672	TCP / HTTP

16.2 Appendix B - Environment Variables Reference

Key environment variables required for deployment (see .env.example for full list):

Variable	Service(s)	Description	Secret?
OS_AUTH_URL	Cloud/Admin	OpenStack Keystone URL	No
OS_USERNAME	Cloud/Admin	OpenStack admin user	Yes
OS_PASSWORD	Cloud/Admin	OpenStack admin password	Yes
POSTGRES_HOST	All	PostgreSQL hostname	No
POSTGRES_PASSWORD	All	Database password	Yes
JWT_SECRET	Gateway/Auth	JWT signing key	Yes
RABBITMQ_URL	Billing/Notif	RabbitMQ connection string	Yes
SMTP_PASS	Notification	SMTP credentials	Yes
REDIS_URL	Gateway/Auth	Redis connection string	No

16.3 Appendix C - Compliance Checklist

- ☐ All secrets stored in Vault/KMS (not in ConfigMaps or env vars)
- ☐ TLS 1.3 enforced on all external endpoints
- ☐ mTLS enabled for all internal service communication

- ☐ Network policies applied and tested in all namespaces
- ☐ Container images scanned and free of CRITICAL CVEs
- ☐ RBAC configured with least-privilege principle
- ☐ Audit logging enabled and shipping to centralized store
- ☐ Backup procedures tested and documented
- ☐ DR failover tested within last 6 months
- ☐ Access reviews completed within last quarter
- ☐ Incident response plan reviewed and updated
- ☐ PII encryption verified at rest and in transit
- ☐ Admin portal isolated from public network

16.4 Appendix D - Document Revision History

Version	Date	Author	Changes
1.0	2026-04-08	Platform Engineering	Initial specification